capital

Custom Auto Link Plugin Development

Software Version 2015.1

February 2016

© 2016 **Mentor Graphics Corporation**
All rights reserved.

This document contains information that is proprietary to Mentor Graphics Corporation. The original recipient of this document may duplicate this document in w hole or in part for internal business purposes only, provided that this entire notice appears in all copies. In duplicating any part of this document, the recipient agrees to make every reasonable effort to prevent the unauthorized use and distribution of the proprietary information.

This document is for information and instruction purposes. Mentor Graphics reserves the right to make changes in specifications and other information contained in this publication without prior notice, and the reader should, in all cases, consult Mentor Graphics to determine whether any changes have been made.

The terms and conditions governing the sale and licensing of Mentor Graphics products are set forth in written agreements between Mentor Graphics and its customers. No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Mentor Graphics whatsoever.

MENTOR GRAPHICS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS MATERIAL INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

MENTOR GRAPHICS SHALL NOT BE LIABLE FOR ANY INCIDENTAL, INDIRECT, SPECIAL, OR CONSEQUENTIAL DAMAGES WHATSOEVER (INCLUDING BUT NOT LIMITED TO LOST PROFITS) ARISING OUT OF OR RELATED TO THIS PUBLICATION OR THE INFORMATION CONTAINED IN IT, EVEN IF MENTOR GRAPHICS CORPORATION HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

RESTRICTED RIGHTS LEGEND 03/97

U.S. Government Restricted Rights. The SOFTWARE and documentation have been developed entirely at private expense and are commercial computer software provided with restricted rights. Use, duplication or disclosure by the U.S. Government or a U.S. Government subcontractor is subject to the restrictions set forth in the license agreement provided with the software pursuant to DFARS 227.7202- 3(a) or as set forth in subparagraph (c)(1) and (2) of the Commercial Computer Software - Restricted Rights clause at FAR 52.227-19, as applicable.

Contractor/manufacturer is:
Mentor Graphics Corporation
8005 S.W. Boeckman Road, Wilsonville, Oregon 97070-7777.
Telephone: 503.685.7000
Toll-Free Telephone: 800.592.2210
Website: www.mentor.com
SupportNet: www.mentor.com/supportnet
Contact Your Technical Writer: www.mentor.com/supportnet/documentation/reply_form.cfm

TRADEMARKS: The trademarks, logos and service marks ("Marks") used herein are the property of Mentor Graphics Corporation or other third parties. No one is permitted to use these Marks without the prior written consent of Mentor Graphics or the respective third-party owner. The use herein of a third- party Mark is not an attempt to indicate Mentor Graphics as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A current list of Mentor Graphics' trademarks may be viewed at:
www.mentor.com/terms_conditions/trademarks.cfm.

End-User License Agreement: You can print a copy of the End-User License Agreement from:
www.mentor.com/terms_conditions/enduser.cfm.

Table Of Contents

Introduction **2**

Interfaces **4**

IXAutoLink 4

IXTopologyAutoLink 5

IXIntegratorAutoLink 5

IXHarnessXCAutoLink 5

Examples **6**

Java class 6

Implements IXHarnessXCAutoLink 10

Class Members 10

Introduction

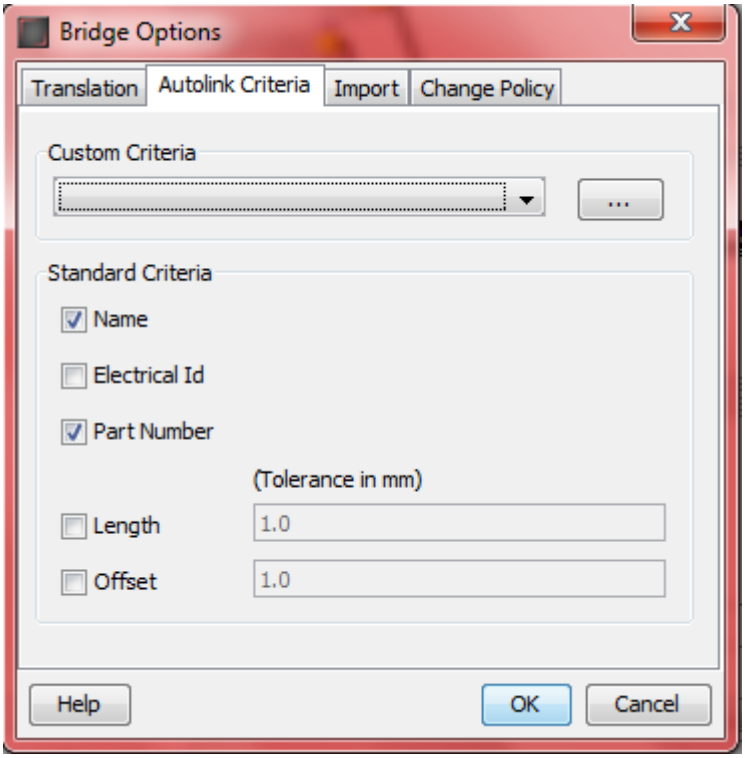
This document describes how to develop a custom auto link plugin.

The criteria available for *Auto Link* action (for Change Manager (CM) in Capital Topology, Integrator and Harness XC applications) are currently *Name, Electrical Id, Part Number, Length and Offset*. Additionally, a plugin may be written and deployed, to determine the linkable design object for a selected external harness object, based on custom criteria.

The following is the functionality overview for a custom auto link plugin:

- If an auto link plugin is selected in the custom criteria section of the *AutoLink Criteria* dialog (see picture 1 below), then the plugin takes the selected external object from CM and the current Capital linked harness as inputs (Alternatively, if the CM query has been modified to show the Capital objects in the dialog, then the selected Capital object and external Harness are taken as inputs). The plugin returns the corresponding Capital design object which is to be linked to the corresponding selected external object (if the query has been modified, then corresponding external object is returned for the selected Capital object). The CM then links the selected/returned external object by the plugin to the corresponding returned/selected Capital design object.
- No other criterion like *Part Number* or *Length* etc is considered for auto link when *Plugin* is selected. Other criteria are considered only if no plugin is selected using the *Plugin* chooser.
- *Plugin* criterion is applicable for all types of external/capital objects supported in the CM.

Picture 1 – AutoLink Criteria dialog with Custom and Standard Criteria



The following are the assumptions / limitations for the proper working of a custom plugin:

- <capital_home>/lib/bridgessdk.jar is in the classpath for successfully compiling autolink plugin
- Autolink plugin has been developed and deployed successfully in the <capital_home>/plugins directory before starting Capital Topology or Integrator or Harness XC applications

Interfaces

IXAutoLink

```
public interface IXAutoLink extends IXPlugin {

      IXObject getLinkableDesignObject(Object x2mlElem, IXHarness
harness);

      Object getLinkableExternalObject(IXObject designObject, Object
harness);

      boolean isAutoLinkableByPlugin(Object object);

}
```

This interface is not to be implemented directly by any auto link plugin. See IXTopologyAutoLink, IXIntegratorAutoLink and IXHarnessXCAutoLink sections for further information.

The following is a description of the methods in this interface.

1. getLinkableDesignObject(object: Object, harness: IXHarness) : IXObject
    a. Parameter object is of type Object and signifies the selected external object in the CM (usually of the type .bridges.x2ml.X2MLElement)
    b. Parameter harness is of the type IXHarness and represents the current design harness linked in CM
    c. The plugin performs various operations as per the business logic on the method parameters
    d. Subsequently, a design object is determined which is to be linked to the selected external object in the CM (parameter object)
    e. This design object of the type IXObject is returned by the plugin
    f. The returned IXObject instance is a part of the method parameter harness
2. getLinkableExternalObject(designObject: IXObject, externalHarness: Object) : Object
    a. Parameter designObject is of type IXObject and signifies the selected object in the CM
    b. Parameter externalHarness is of the type Object and represents the current external harness selected in the CM (usually of the type .bridges.x2ml.Harness)
    c. The plugin performs various operations as per the business logic on the method parameters
    d. Subsequently, an object from the external harness is determined which is to be linked to the selected object in the CM (parameter designObject)

e. This external object is returned with type Object (usually of type .bridges.x2ml.X2MLElement)

3. isAutoLinkableByPlugin(object: Object) : boolean

   a. Parameter object is of type Object and signifies an external object or a object selected in CM

   b. Before invoking methods 1 or 2 above, first calls this method of the plugin

   c. This method enables the plugin to choose what type of objects it can handle. For example, if the plugin has been written to auto link only connectors, then this method should return true only for connector objects selected in the CM. For all other object types, it should return false

   d. Method 1 or 2 of the plugin is called by only if method 3 returns true. Otherwise, the plugin is not used and the usual auto link with standard criteria happens

IXTopologyAutoLink

IXTopologyAutoLink extends IXAutoLink and does not have any methods by itself. Any auto link plugin which is intended to work with CM in Capital Topology application has to implement this interface.

IXIntegratorAutoLink

IXIntegratorAutoLink extends IXAutoLink and does not have any methods by itself. Any auto link plugin which is intended to work with CM in Integrator application has to implement this interface.

IXHarnessXCAutoLink

IXHarnessXCAutoLink extends IXAutoLink and does not have any methods by itself. Any auto link plugin which is intended to work with CM in Harness XC application has to implement this interface.

Examples

In this section, we shall see the example of HarnessXCAutoLink.java, a Java plugin which determines auto linkable objects for selected objects in Harness XC CM, based on the following logic:

- Check if the selected object in CM for auto link, has a user property with name “AUTOLINK” and note its value
- Find object of the same type in the passed harness, which also has a user property with name “AUTOLINK” and with same value as for the selected object in CM
- Return this object in the passed harness, as the auto linkable object for the selected object in CM

Java class

```
public class HarnessXCAutoLink implements IXHarnessXCAutoLink
{
     private static final String AUTOLINK_PROP_NAME = "AUTOLINK";

     public String getDescription() {
          return "Finds the autolinkable object for a selected
          object in Harness XC Change Manager";
     }

     public String getName() {
          return "HarnessXCAutoLink";
     }

     public String getVersion() {
          return "1.0";
     }

     public IXObject getLinkableDesignObject(Object x2mlElem,
                                             IXHarness harness)
     {
          if(harness != null && x2mlElem instanceof X2MLElement)
          {
               Collection<UserProperty> userProps =
                    ((X2MLElement)x2mlElem).getUserProperties();
               String autoLinkVal = null;
               for(UserProperty prop : userProps) {

     if(prop.getName().equalsIgnoreCase(AUTOLINK_PROP_NAME))
```

```
{
                    autoLinkVal = prop.getValue();
                    break;
}
          }
          if(autoLinkVal != null &&
                         !"".equals(autoLinkVal.trim()))
          {
               Set desObjs = null;
               if(x2mlElem instanceof Bundle) {
                    desObjs = harness.getBundles();
               } else if(x2mlElem instanceof Splice) {
                    desObjs = harness.getSplices();
               } else if(x2mlElem instanceof Fixing) {
                    desObjs = new HashSet();
                    desObjs.addAll(harness.getClips());
                    desObjs.addAll(harness.getGrommets());
               desObjs.addAll(harness.getOtherComponents());
               } else if(x2mlElem instanceof Protection) {
                    Set<IXInsulationRun> insRuns =
                    harness.getInsulationRuns();
                    desObjs = new HashSet();
                    for(IXInsulationRun insRun : insRuns)
                    {
                    desObjs.addAll(insRun.getInsulations());
                    }
                    desObjs.addAll(harness.getSpotTapes());
               } else if(x2mlElem instanceof Connector) {
                    desObjs = harness.getConnectors();
               } else if(x2mlElem instanceof Wire) {
                    desObjs = harness.getWires();
               } else if(x2mlElem instanceof Multicore) {
                    desObjs = harness.getMulticores();
               }

               IXObject reqObj = null;
               if(desObjs != null)
               {
                    Iterator iter = desObjs.iterator();
                    while(iter.hasNext())
                    {
                         Object obj = iter.next();
                         if(obj instanceof IXObject &&
                         autoLinkVal.equals(((IXObject)obj)
                         .getProperty(AUTOLINK_PROP_NAME)))
                         {
```

```
                                if(reqObj == null)
                                {
                                     reqObj = (IXObject)obj;
                                } else
                                {
                                     return null;
                                }
                           }
                      }
                 }
                 return reqObj;
            }
       }
       return null;
  }

  public Object getLinkableExternalObject(IXObject
                                designObject, Object harness)
  {
       X2MLElement reqObj = null;
       if(harness instanceof IWiringContainer && designObject
                                     instanceof IXObject)
       {
            String autoLinkVal =
                 designObject.getProperty(AUTOLINK_PROP_NAME);
            if(autoLinkVal != null &&
                           !"".equals(autoLinkVal.trim()))
            {
                 Collection harObjs = null;
                 if(designObject instanceof IXBundle)
                 {
                      harObjs = ((IWiringContainer)harness).
                 getElementsByType(Bundle.getElementName());
                 } else if(designObject instanceof IXSplice)
                 {
                      harObjs = ((IWiringContainer)harness).
                 getElementsByType(Splice.getElementName());
                 } else if(designObject instanceof IXClip ||
                      designObject instanceof IXGrommet ||
                 designObject instanceof IXOtherComponent)
                 {
                      harObjs = ((IWiringContainer)harness).
                 getElementsByType(Fixing.getElementName());
                 } else if(designObject instanceof
                 IXInsulation ||
                      designObject instanceof IXSpotTape)
                 {
```

```
                    harObjs = ((IWiringContainer)harness).
          getElementsByType(Protection.getElementName());
               } else if(designObject instanceof
                                        IXConnector)
               {
                    harObjs = ((IWiringContainer)harness).
          getElementsByType(Connector.getElementName());
               } else if(designObject instanceof IXWire)
               {
                    harObjs = ((IWiringContainer)harness).
               getElementsByType(Wire.getElementName());
               } else if(designObject instanceof
                                        IXMulticore)
               {
                    harObjs = ((IWiringContainer)harness).
          getElementsByType(Multicore.getElementName());
               }
               if(harObjs != null)
               {
                    Iterator iter = harObjs.iterator();
                    while(iter.hasNext())
                    {
                         Object obj = iter.next();
                         if(obj instanceof X2MLElement)
                         {
                              Collection<UserProperty> props
               = ((X2MLElement)obj).getUserProperties();
                              for(UserProperty prop : props)
                              {

if(AUTOLINK_PROP_NAME.equalsIgnoreCase(prop.getName()) &&

                    autoLinkVal.equals(prop.getValue()))
{
                         if(reqObj == null)
                         {
                              reqObj = (X2MLElement)obj;
                              break;
                         } else
                         {
                              // ambiguous match!
                              return null;
                         }
}
                              }
                         }
```

```
                        }
                    }
                }
            }
            return reqObj;
        }

        public boolean isAutoLinkableByPlugin(Object object)
        {
            return (object instanceof IXObject || object instanceof
                                                    X2MLElement);
        }
}
```

Implements IXHarnessXCAutoLink

As discussed in the previous section, this class implements the IXHarnessXCAutoLink interface. This enables this sample application to be used from Harness XC CM for custom auto link. If a similar functionality is desired in Integrator CM, then the plugin should implement IXIntegratorAutoLink interface

Class Members

1. AUTOLINK_PROP_NAME attribute: This is the constant String with value “AUTOLINK”.
2. getDescription method: This method returns a description string about what the current plugin does (This method is from the IXPlugin interface, which the IXAutoLink extends).
3. getName method: This returns the name of the current plugin (This method is from the IXPlugin interface, which the IXAutoLink extends).
4. getVersion method: This returns the version of the current plugin (This method is from the IXPlugin interface, which the IXAutoLink extends).
5. getLinkableDesignObject method: This method determines and returns the linkable design object for a given object in the external harness (design object which has the same value for the user property “AUTOLINK”, as for the passed external object)
6. getLinkableExternalObject method: This method determines and returns the linkable external object for a given object in the linked Harness XC design (external object which has the same value for the user property “AUTOLINK”, as for the passed design object)
7. isAutoLinkableByPlugin method: This method determines what kinds of objects are handled by this plugin. Any object which is either of type IXObject (design object) or of type .bridges.x2ml.X2MLElement (external harness object) is handled by this plugin. Suppose, if the plugin desires to handle only bundles, then the code in this method would resemble the following:

    return (object instanceof IXBundle || object instanceof Bundle);